Deep Reinforcement Learning

Temporal Difference Learning & Q Learning

Prof. Dr. Sebastian Peitz

Chair of Safe Autonomous Systems, TU Dortmund

Summer term 2026

Content

Content

- The TD learning concept
- GLIE
- SARSA
- Q-learning

The TD learning concept

Temporal-difference learning and the previous methods

The main aspects of our previous two categories of methods:

1. Monte Carlo (MC): **learning from experience**, possibly without knowledge of a model, e.g., *MC prediction* with exploring starts:

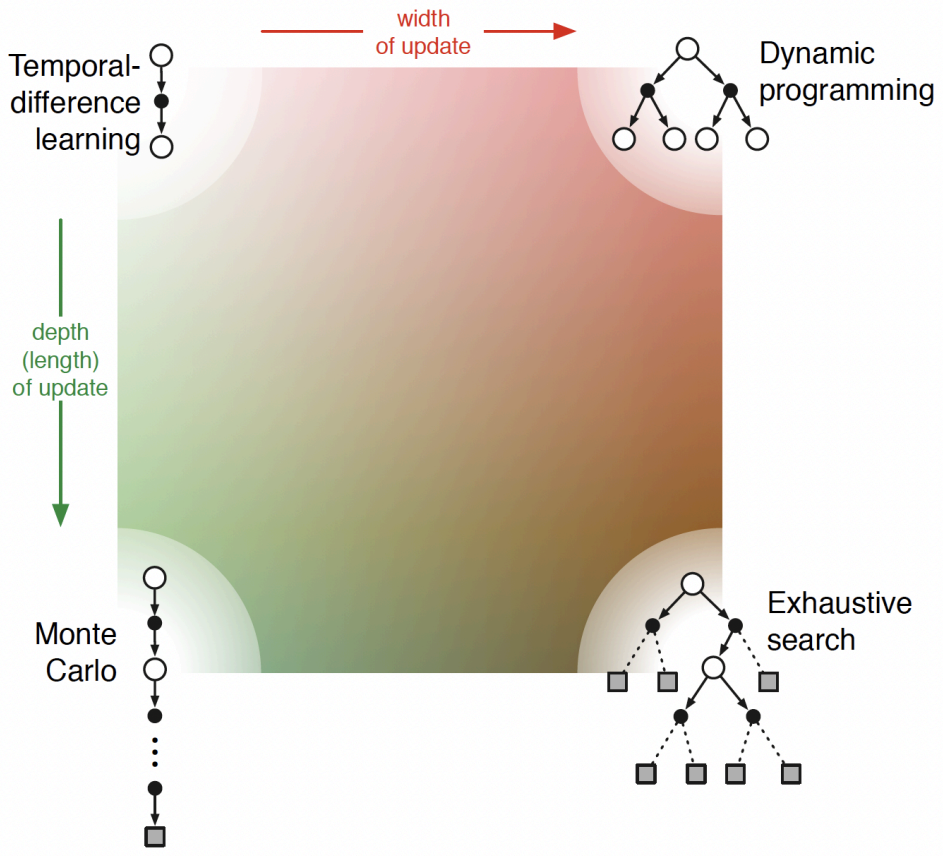
$$Q(s_t, a_t) = Q(s_t, a_t) + \frac{1}{n(s_t, a_t)}\left[g - Q(s_t, a_t)\right].$$

2. Dynamic programming (DP): **bootstrapping** ⇒ updating estimates based on estimates, e.g., *iterative policy evaluation*:

$$V(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{s' \in \mathcal{S}} p(s'|s, a)\left[r + \gamma V(s')\right].$$

The **concept of temporal difference (TD) learning** is to combine both:

1. *Model-free* prediction and control in unknown MDPs.
2. Updates policy evaluation and improvement in an online fashion (i.e., not per episode) by *bootstrapping*.



(*Sutton and Barto 1998*, *Figure 8.11*)

💡 We are still considering finite MDPs here.

The general TD prediction update

Recall the **incremental update** of the type we discussed for multi-armed bandits or for MC control $\Rightarrow$ **NewEstimate** $=$ **OldEstimate** $+ \mathit{StepSize}\,[$**Target** $-$ **OldEstimate**$]$:

$$V(s_t) = V(s_t) + \alpha\left[g_t - V(s_t)\right]. \tag{1}$$

- $\alpha \in (0, 1)$ is the forget factor / step size.
- g_t is the target of the incremental update rule.
- To execute (1), we have to wait until the end of the episode to get g_t.

One-step TD / TD(0) update

$$V(s_t) = V(s_t) + \alpha\left[r_t + \gamma V(s_{t+1}) - V(s_t)\right]. \tag{2}$$

- Here, the **TD target** is $r_t + \gamma V(s_{t+1})$.
- TD is *bootstrapping*: estimate $V(s_t)$ based on $V(s_{t+1})$.
- *Delay time of one* time step; no need to wait until the end of the episode.

Algorithmic implementation: TD-based prediction

Algorithm: TD-based prediction.
Parameters: Step size $\alpha \in (0, 1)$

Initialize: $V(s)$ arbitrarily for $s \in \mathcal{S}$, $V(\textbf{terminal}) = 0$

for $k = 1, 2, \ldots, K$ episodes:
    for $t = 0, 1, \ldots, T$:
        Sample action $a_t \sim \pi(a|s)$ and apply
        Observe s_{t+1} and r_t
        $V(s_t) = V(s_t) + \alpha\left[r_t + \gamma V(s_{t+1}) - V(s_t)\right]$ (Eq. (2))
        if s_{t+1} is terminal **then** STOP

💡 The algorithm can directly be applied to the prediction of action-value functions.

The TD error and its relation to the MC error

Using the target, we can define the **TD error*** $\delta \Rightarrow$ the expression inside the bracket of Eq. (2) we're using to improve our estimate:

$$\delta_t = \textbf{target} - V(s_t) = r_t + \gamma V(s_{t+1}) - V(s_t).$$

Batch mode: Let's assume that the TD(0) estimate of $V(s)$ does not change within an episode, but that we apply all updates simulatenously once the episode is finished (exactly as we need to do in MC prediction):

$$\begin{aligned}
\underbrace{g_t - V(s_t)}_{\text{MC error}} &= r_t + \gamma g_{t+1} - V(s_t) + \gamma V(s_{t+1}) - \gamma V(s_{t+1}) \\
&= \delta_t + \gamma(\underbrace{g_{t+1} - V(s_{t+1})}_{\text{MC error}}) = \delta_t + \gamma \delta_{t+1} + \gamma^2 (g_{t+2} - V(s_{t+2})) \\
&= \delta_t + \gamma \delta_{t+1} + \gamma^2 \delta_{t+2} + \gamma^3 (g_{t+3} - V(s_{t+3})) = \ldots = \sum_{k=t}^{T} \gamma^{k-t} \delta_k.
\end{aligned}$$

- The MC error is the discounted sum of TD errors in this case.
- If $V(s)$ is updated during an episode (as is common in TD(0)), the above identity only holds approximately.

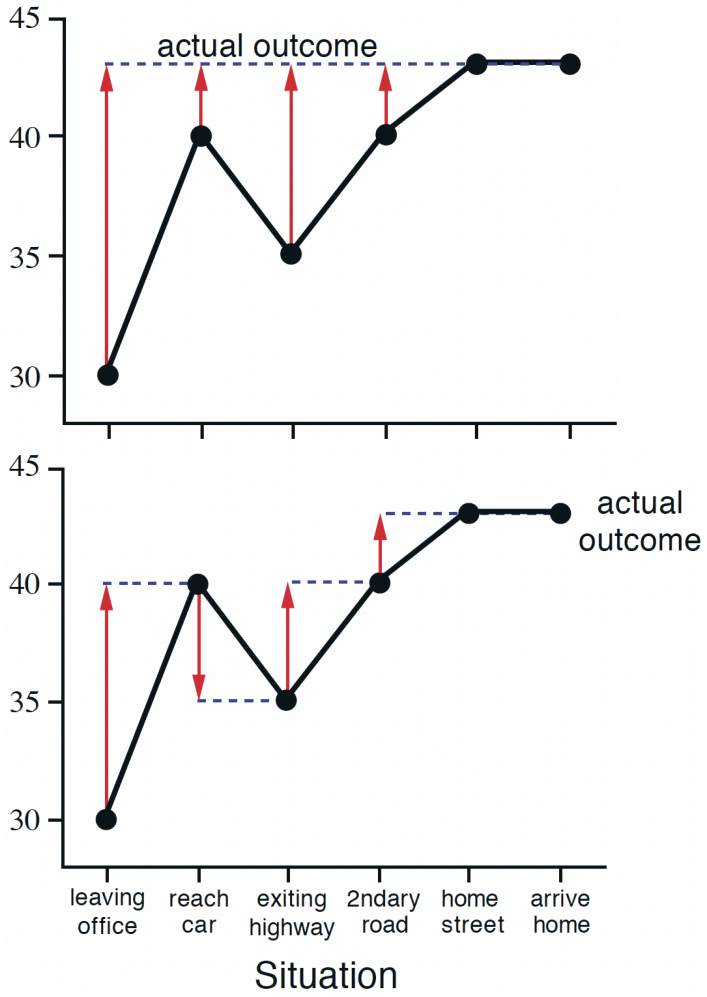
* For sufficiently small step size α, it can be shown that the TD error conveges to zero in batch mode (Sutton and Barto 1998).

The *driving home* example

Imagine you want to predict your time to get home after work, starting with an estimate of 30 minutes.

State	Elapsed Time (minutes)	Predicted Time to Go	Predicted Total Time
leaving office, friday at 6	0	30	30
reach car, raining	5	35	40
exiting highway	20	15	35
2ndary road, behind truck	30	10	40
entering home street	40	3	43
arrive home	43	0	43

Overview of elapsed time and your predictions of time to go and total time (Sutton and Barto 1998, Example 6.1)



MC vs. TD estimate

- In MC, we estimate the returns g_t **after** we have reached home ⇒ it is hard to assess where the deviation was caused.
- In TD, we can immediately shift our estimate in each time step.

- With TD, we can even learn from continuing tasks!

The batch training AB example

asf

Content

- Recall GPI
- GLIE
- SARSA (on-policy)
- Q-learning (off-policy)

Greedy in the limit with infinite exploration (GLIE)

A learning policy π is called GLIE if it satisfies the following two properties:

- If a state is visited inifinitely often, then each action is choosen inifintely often:

$$\lim_{i \to \infty} \pi_i(a|s) = 1 \quad \forall \{s \in \mathcal{S}, a \in \mathcal{A}\}$$

- In the limit ($i \to \infty$) the learning policy is greedy with respect to the learned action value:

$$\lim_{i \to \infty} \pi_i(a|s) = \pi(s) = \arg\max_{a \in \mathcal{A}} Q(s, a) \quad \forall s \in \mathcal{S}$$

GLIE Monte Carlo control

MC-based contro using ϵ-greedy exploration is GLIE, if ϵ is decreased at a rate

$$\epsilon_i = \frac{1}{i}$$

with i being the increasing episode index. In this case,

$$\hat{Q}(s, a) = Q^*(s, a)$$

follows.

SARSA

- **S**tate-**A**ction-**R**eward-Next-**S**tate-Next-**A**ction
- on-policy

Algorithm: SARSA.
initialize

- $Q(s, a)$ arbitrarily for $s \in \mathcal{S}, a \in \mathcal{A}$
- $Q(\text{terminal-state}, \cdot) = 0$
- $\pi = \epsilon\text{-greedy}(Q)$

for $j = 1, 2, \ldots, J$ episodes:
  Initialize $s_t \leftarrow s_0, t \leftarrow 0$
  while s_t is not terminal:
    Take action $a_t \sim \pi(s_t)$ and observe (r_t, s_{t+1})
    Select $a_{t+1} \sim \pi(s_{t+1})$
    Update Q given $(s_t, a_t, r_t, s_{t+1}, a_{t+1})$:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)\right]$$

    Update policy $\pi = \epsilon\text{-greedy}(Q)$
    $t \leftarrow t + 1$

Convergence of SARSA

[Based on Marius Lindauer's lecture]

SARSA for finite-state and finite-action MDPs converges to the optimal action-value, $Q(s, a) \to Q^*(s, a)$, under the following conditions:

1. The policy sequence $\pi_t(a \mid s)$ satisfies the condition of GLIE

2. The step-sizes α_t satisfy the Robbins-Munro sequence such that

$$\sum_{t=1}^{\infty} \alpha_t = \infty$$
$$\sum_{t=1}^{\infty} \alpha_t^2 < \infty$$

For example, $\alpha_t = \frac{1}{t}$ satisfies the above condition.

Q-Learning

- SARSA estimates Q of the current policy
- Q-learning is similar but directly estimates Q^*
- off-policy update, since the optimal action-value function is updated independent of a given behavior policy

Algorithm: Q-Learning.
initialize

- $Q(s, a)$ arbitrarily for $s \in \mathcal{S}, a \in \mathcal{A}$
- $Q(\text{terminal-state}, \cdot) = 0$
- $\pi = \epsilon\text{-greedy}(Q)$

for $j = 1, 2, \ldots, J$ episodes:
  Initialize $s_t \leftarrow s_0, t \leftarrow 0$
  while s_t is not terminal:
    Take action $a_t \sim \pi(s_t)$ and observe (r_t, s_{t+1})
    Select $a_{t+1} \sim \pi(s_{t+1})$
    Update Q given $(s_t, a_t, r_t, s_{t+1}, a_{t+1})$:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right]$$

    Update policy $\pi = \epsilon$-greedy(Q)
    $t \leftarrow t + 1$

References

Sutton, R. S., and A. G. Barto. 1998. *Reinforcement Learning: An Introduction*. MIT Press.